RoomRover -
Smart IoT Based Navigation

Submitted in partial fulfilment of the requirements of the degree of

BACHELOR OF ENGINEERING

by

Maitreyi Phadke: 22106007

Gauri Ramekar: 22106068

Arya Raut: 22106075

Chinmay Sawant: 22106017

Guide:

Prof. Sayali P. Badhan

Department of Computer Science & Engineering

(Artificial Intelligence & Machine Learning)

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

(2025-2026)

A.P. SHAH INSTITUTE OF TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled **"RoomRover - Smart IoT Based Navigation"** is a bonafide work of **Maitreyi Phadke(22106007), Gauri Ramekar(22106068), Arya Raut(22106075), Chinmay Sawant(22106017)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning).**

Prof. Sayali P. Badhan Guide	Prof. Sayali P. Badhan Project Coordinator
Prof. Dr. Jaya Gupta Head of Department	Dr. Uttam D. Kolekar Principal

A.P. SHAH INSTITUTE OF TECHNOLOGY

Project Report Approval for B.E.

This project report entitled **"RoomRover - Smart IoT Based Navigation"** by **"*Maitreyi Phadke, Gauri Ramekar, Arya Raut, Chinmay Sawant*"** is approved for the degree of ***Bachelor of Engineering*** in ***Computer Science & Engineering (Artificial Intelligence & Machine Learning), 2025-26***.

Examiner Name	Signature
1._____	_____
2._____	_____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Maitreyi Phadke & 22106007)

(Gauri Ramekar & 22106068)

(Arya Raut & 22106075)

(Chinmay Sawant & 22106017)

Date:

Abstract

The **"RoomRover"** project introduces an integrated intelligent system that combines autonomous robotics, Internet of Things (IoT), and Artificial Intelligence (AI) to promote sustainability and innovation. The system features a dual-tier architecture: a physical tier comprising an ESP32-based mobile robot for autonomous navigation, obstacle avoidance, and hazard detection, and a cognitive tier consisting of a local voice-enabled RAG (Retrieval-Augmented Generation) chatbot. The robot transmits real-time sensor data and alerts to the Blynk IoT platform, while the Python-based chatbot processes voice commands for navigation simulation and answers user queries by retrieving information from a local knowledge base. This synergistic approach creates a smart, interactive environment for monitoring and assistance, demonstrating a practical application of collaborative AI and IoT technologies. In alignment with the United Nations Sustainable Development Goals, the RoomRover project supports **Goal 3: Good Health and Well-being** by enhancing indoor safety and promoting healthy living environments, **Goal 9: Industry, Innovation and Infrastructure** by fostering innovation through the integration of intelligent robotics and IoT systems, and **Goal 11: Sustainable Cities and Communities** by contributing to the development of inclusive, safe, and resilient smart indoor spaces. The system ultimately aims to provide a scalable and efficient solution for remote monitoring, voice-controlled assistance, and autonomous hazard response in modern sustainable environments.

Keywords:

IoT, Autonomous Robot, ESP32, Voice Assistant, RAG, Blynk Cloud, Obstacle Avoidance, Smoke Detection, Speech-to-Text, AI

CONTENTS

1. Introduction…….....……..……... 6
2. Literature Survey….……...…….…….....……7
3. Limitation of Existing System…….…….....8
4. Problem Statement, Objectives and Scope….....….……..9
5. Proposed System…….....11
6. Experimental Setup…….....19
7. Project Planning……...……...22
8. Implementation……...24
9. References…….....26

LIST OF FIGURES

5.1. Architecture Diagram for RoomRover – Smart IoT Based Navigation……11
5.2. RoomRover Dataflow Diagram …….....11
5.3. RAG Dataflow Diagram ……...12
5.4. RoomRover Use Case Diagram…..……...12
5.5. RAG Use Case Diagram …..…….....13
5.6. RoomRover Sequence Diagram ……...13
5.7. RAG Sequence Diagram ……14
5.8. RAG Activity Diagram ……14
5.9. RoomRover Activity Diagram ……15
7.1. July (Gantt Chart) ……22
7.2. August (Gantt Chart) ……22
7.3. September (Gantt Chart) ……23
7.4. October (Gantt Chart) ……23
8.1 Blynk Console…… 24
8.2 Floor Plan…...…… 24
8.3 Chatbot UI……...25

LIST OF TABLES

2.1. Literature Survey ……..……..7

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ESP32	A series of low-cost, low-power system on a chip microcontrollers
GPT-2	Generative Pre-trained Transformer 2
IDE	Integrated Development Environment
IoT	Internet of Things
RAG	Retrieval-Augmented Generation
STT	Speech-to-Text
TTS	Text-to-Speech
Wi-Fi	Wireless Fidelity

Chapter 1
Introduction

In the rapidly evolving landscape of smart technology, the integration of autonomous systems into daily environments has become a critical area of innovation. While standalone devices like robotic vacuums and stationary voice assistants have gained popularity, they operate in isolated ecosystems, lacking the ability to combine physical mobility with intelligent, context-aware interaction. Traditional remote monitoring systems often require constant human oversight and lack autonomous decision-making, while voice assistants are confined to their fixed locations, unable to physically interact with or navigate their surroundings. This creates a significant gap between passive environmental sensing and active, intelligent engagement. With advancements in artificial intelligence, IoT, and embedded systems, there is now a powerful opportunity to bridge this divide. Technologies like Retrieval-Augmented Generation (RAG) are revolutionizing natural language understanding by allowing AI models to pull information from custom knowledge bases, providing more accurate and relevant responses. This project, RoomRover, proposes a novel system that merges these disparate fields into a single, cohesive unit. It combines a physically mobile, sensor-equipped robot with an intelligent voice-based chatbot powered by a RAG model. Unlike conventional systems that are either physically mobile or conversationally intelligent, RoomRover is both. The system is designed as a dual-architecture solution: an ESP32-based autonomous robot for navigating and sensing the physical world, and a sophisticated AI chatbot that serves as the cognitive core for interaction and information retrieval. The primary motivation for this project is to create a versatile and accessible platform that demonstrates the power of combining mobility, IoT connectivity, and advanced AI. By developing a system that can autonomously monitor its environment for hazards while simultaneously responding to complex voice-based queries, RoomRover serves as a proof-of-concept for the next generation of integrated smart systems for homes, offices, and educational settings.

Chapter 2

Literature Survey

Year	Title	Domain	Algorithm/Methodology	Contribution	Gap
2025	A Survey on Autonomous Navigation for Mobile Robots: From Traditional Techniques to Deep Learning and Large Language Models	Robotics / AI	Review of A*, D*, SLAM, and deep learning methods	Summarizes robot navigation evolution and rise of AI-based models	Limited focus on integrating voice commands
2022	Sensor Based Algorithm for Self-Navigating Robot Using IoT	IoT / Robotics	Uses ultrasonic and IR sensors for obstacle avoidance with IoT control	Simple, effective sensor-fusion for basic navigation	Only reactive navigation; lacks mapping or planning
2021	A SLAM-Based Indoor Navigation System for Service Robots	Robotics / Computer Vision	Applies SLAM to map indoor environments and aid navigation	Provides robust mapping and path-planning approach	High computation and sensor cost; unsuitable for low-cost bots
2020	IoT-Based Fire Detection and Alert System	IoT / Safety Systems	Smoke & temperature sensors send alerts to cloud	Reliable system for remote hazard alerts	Stationary; cannot detect hazard source

Chapter 3

Limitations of Existing Systems

Although several methods exist for autonomous navigation and remote sensing, many of them face significant challenges when aiming for a truly integrated and intelligent system. Traditional robotic systems are often functionally siloed, while newer AI-based approaches can be complex or narrowly focused. The major limitations identified from the literature are as follows:

Fragmented Functionality Existing systems are typically specialized for either navigation or stationary sensing. For instance, IoT-based alert systems are effective for hazard detection but are immobile, while basic navigating robots lack sophisticated sensory and communication capabilities. This creates a disconnect between environmental awareness and physical action.

High Cost and Complexity of Advanced Navigation Robust navigation solutions like SLAM provide excellent mapping and localization but come with high computational overhead and often require expensive sensors such as LiDAR. This makes them less accessible for cost-effective, scalable applications in consumer or educational environments.

Lack of Intelligent, High-Level Control Most existing low-cost robots rely on simple, reactive algorithms (e.g., obstacle avoidance) or require manual control through a mobile app. They lack the ability to understand and execute high-level, natural language commands, which limits their utility and makes user interaction less intuitive.

Reactive vs. Interactive Systems Many sensor-based robots are purely reactive and cannot be tasked with complex goals. Their interaction model is limited to avoiding objects rather than engaging with a user's informational or navigational queries. This prevents them from serving as true assistants.

Limited Integration of AI and IoT While many projects use either AI or IoT, the deep integration of both is less common. IoT platforms are often used simply for data logging or remote control, without leveraging AI to interpret data, make intelligent decisions, or facilitate natural user interaction through a cognitive interface like a chatbot.

Chapter 4

Problem Statement, Objectives and Scope

Problem Statement

Indoor safety and real-time monitoring remain major challenges in homes, offices, and laboratories. Traditional safety solutions, like static smoke detectors or manual patrols, fail to provide real-time mobility, obstacle-aware navigation, and integrated alert systems. Additionally, human assistance in retrieving information or guiding users in large facilities (e.g., campuses, labs, warehouses) can be time-consuming and inconsistent. There is a growing need for a smart autonomous system that combines robotic sensing, IoT monitoring, and intelligent voice interaction, allowing the environment to be monitored proactively while providing real-time navigation assistance and interactive responses. The lack of an integrated solution combining autonomous robotics, IoT cloud monitoring, and voice-based AI interaction limits safety, efficiency, and user engagement. RoomRover aims to bridge this gap by offering autonomous movement, smoke detection, IoT-based alerts, and a voice-enabled RAG assistant capable of simulated navigation and information retrieval.

Objectives

1. Design an Autonomous Robotic System:

- Develop a mobile robot using ESP32 microcontroller with ultrasonic sensors for obstacle detection and DC motors for omnidirectional movement.
- Implement safe navigation strategies that stop or reroute the robot to prevent collisions in real-time.

2. Integrate Smoke Detection and Alert Mechanism:

- Incorporate an MQ2 smoke sensor for real-time smoke detection.
- Trigger visual/audio alerts (buzzer, Blynk notifications) when hazardous conditions are detected.

3. Enable IoT-Based Monitoring and Control:

- Connect ESP32 to Blynk Cloud for remote monitoring and control.
- Provide mobile app access to view live sensor data, send movement commands, and receive alerts from anywhere.

4. Develop Voice-Enabled RAG Desktop Assistant:

- Implement a Tkinter-based GUI capable of taking voice input and providing natural-language responses using RAG (Retrieval-Augmented Generation).
- Support navigation simulation, allowing the user to request room guidance (e.g., "Take me to room 204") and receive step-by-step simulated navigation feedback.

5. Ensure Modular, Scalable, and Offline Operation:

- Design the system to operate offline for AI processing while remaining modular for future enhancements.
- Allow seamless scaling to integrate real robot navigation based on voice commands, new sensors, or expanded knowledge bases.

Scope

The RoomRover project lies at the intersection of IoT, robotics, and AI-powered voice interaction. The system is capable of:

- Autonomous navigation within indoor environments, avoiding obstacles using ultrasonic sensors.
- Smoke detection and alerting, providing proactive safety monitoring.
- IoT-enabled control and monitoring, allowing remote access via Blynk Cloud.
- Voice-driven intelligent interaction, answering queries and simulating indoor navigation using a local text-based RAG knowledge base.

Chapter 5

Proposed System

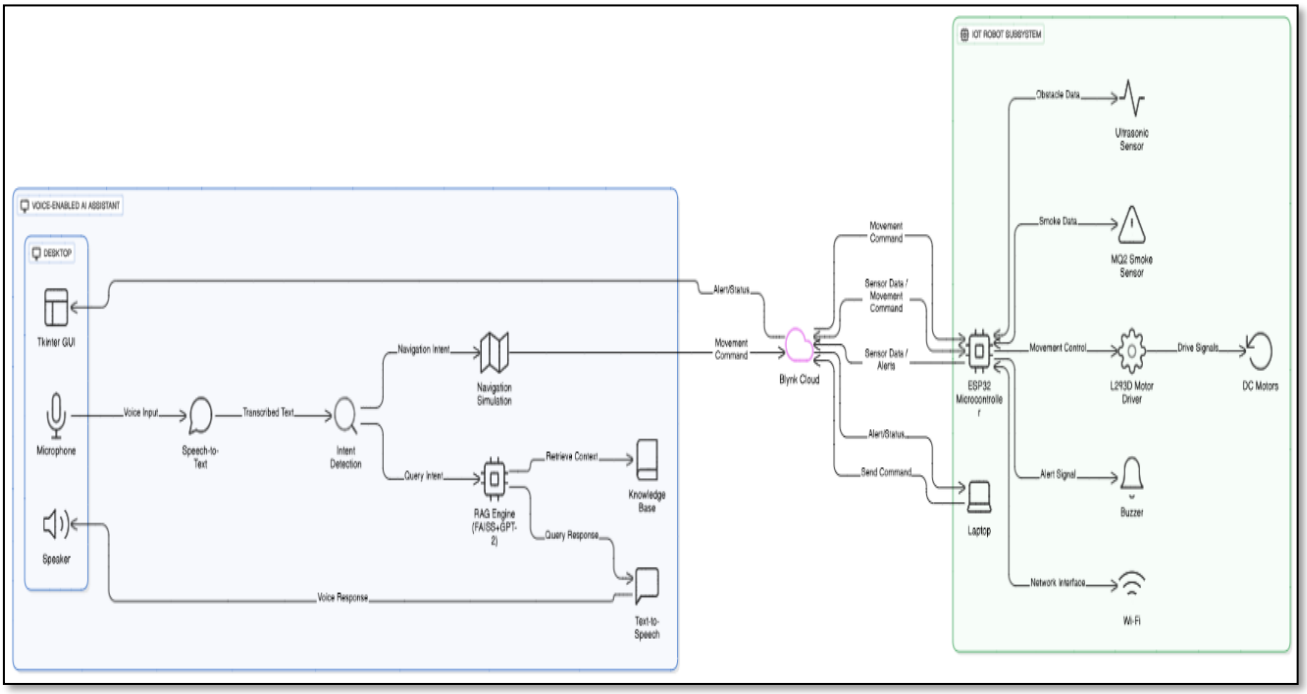

Figure 5.1 Architecture Diagram for RoomRover - Smart IoT Based Navigation

Data Flow Diagrams:

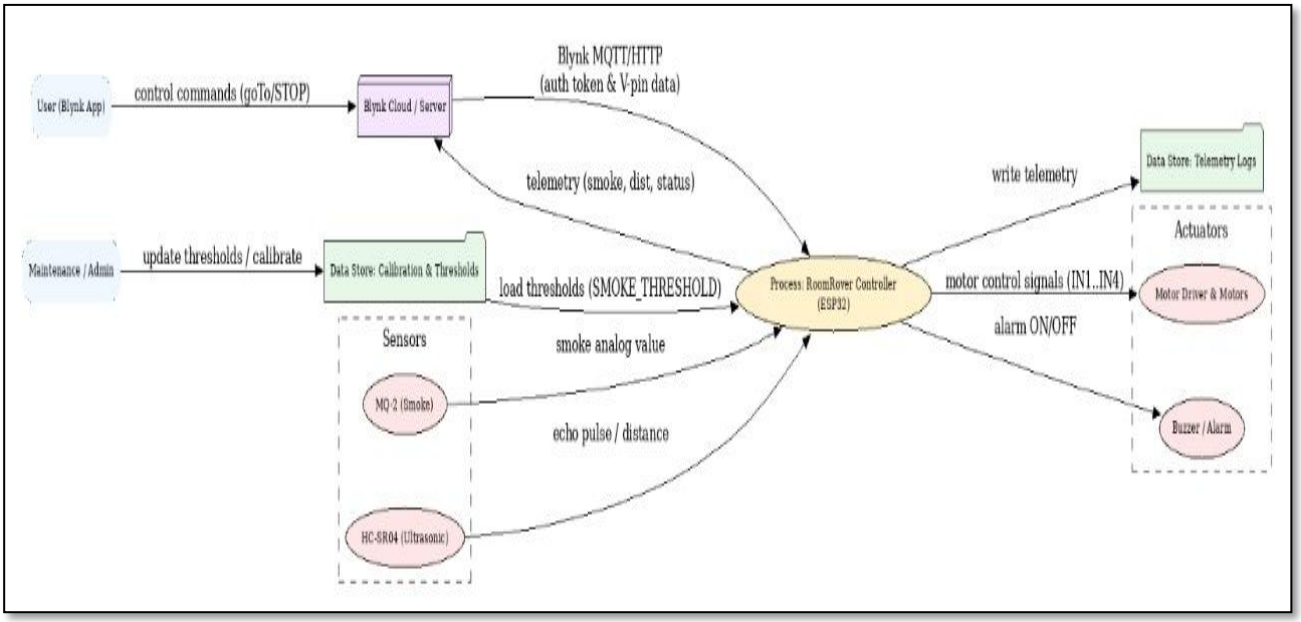

Figure 5.2 RoomRover Dataflow Diagram

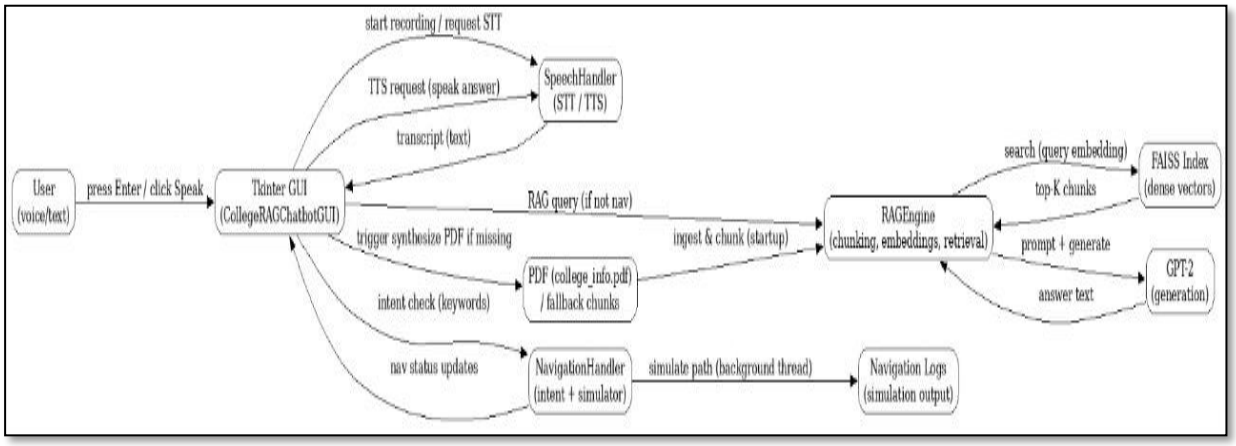

Figure 5.3 RAG Dataflow Diagram

Use Case Diagram:

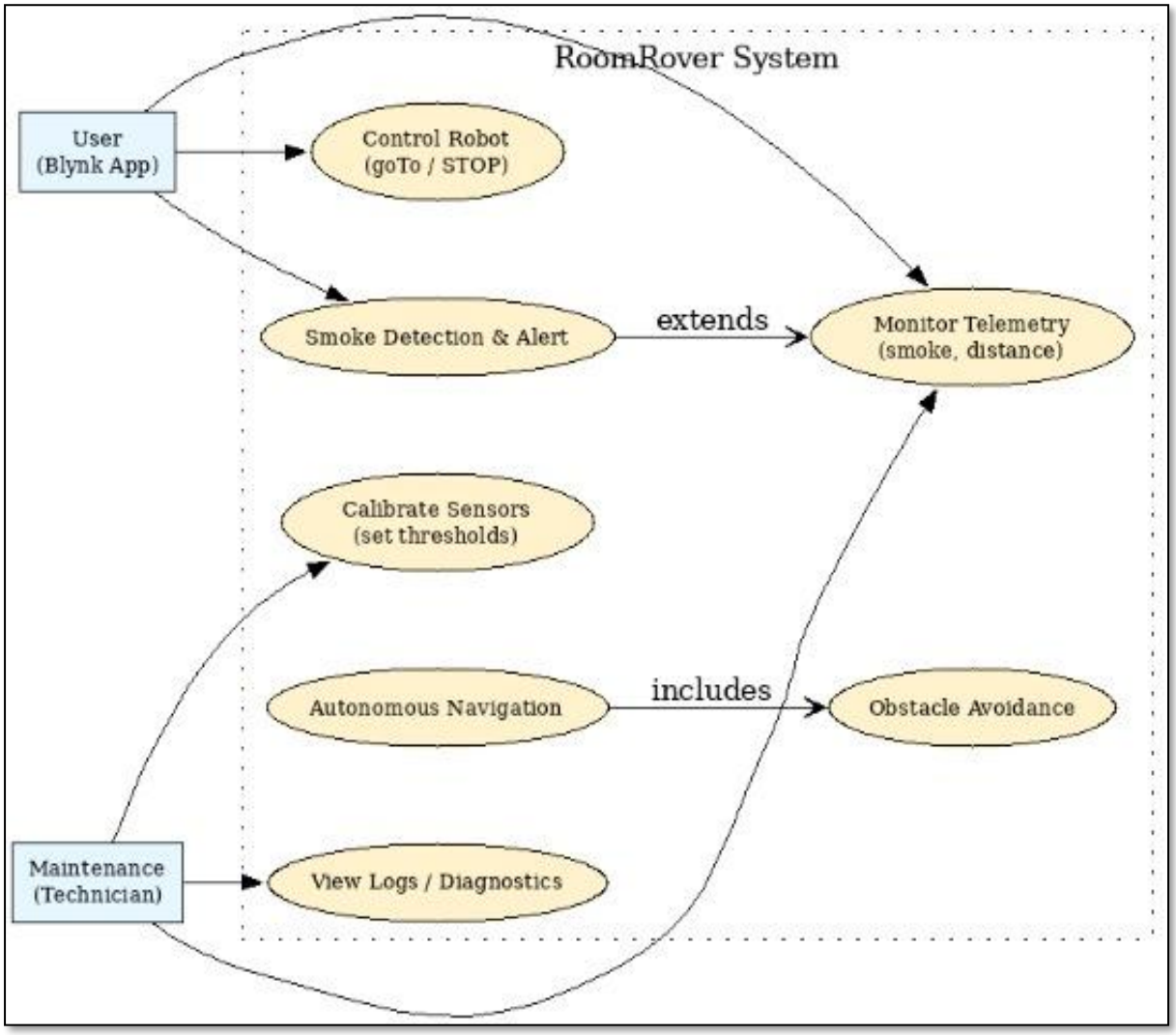

Figure 5.4 RoomRover UseCase Diagram

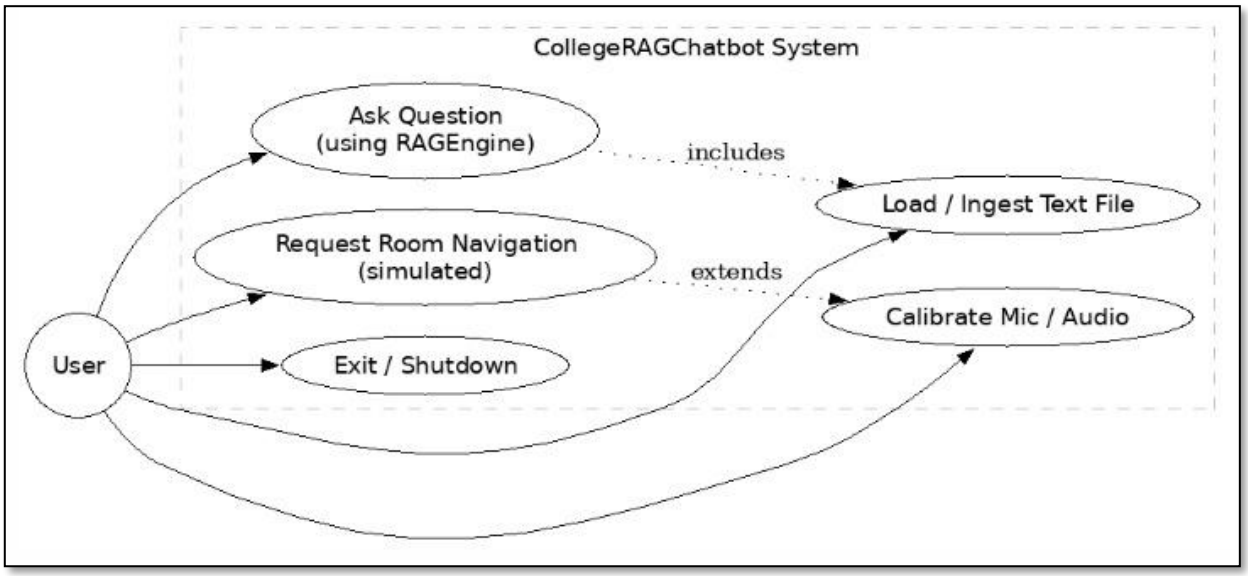

Figure 5.5 RAG UseCase Diagram

Sequence Diagram

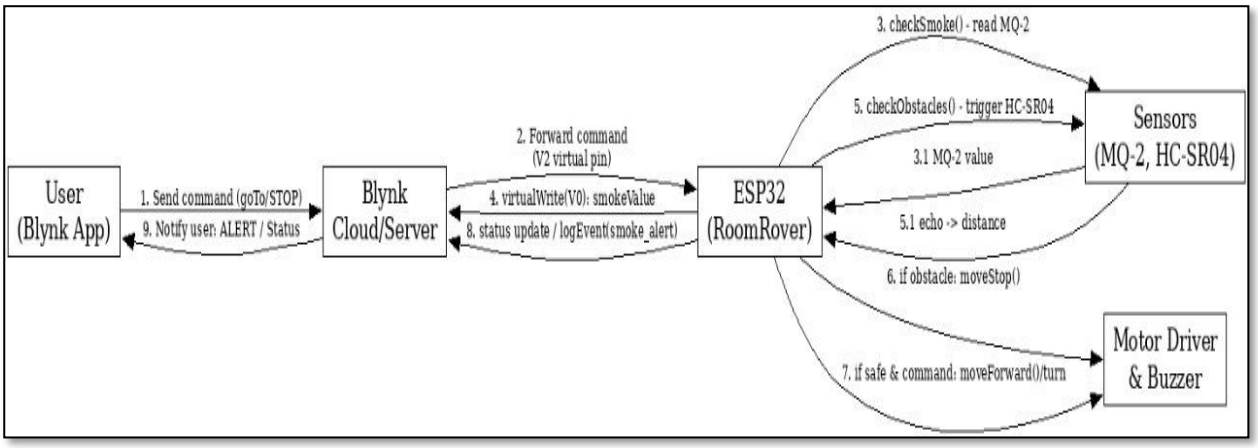

Figure 5.6 RoomRover Sequence Diagram

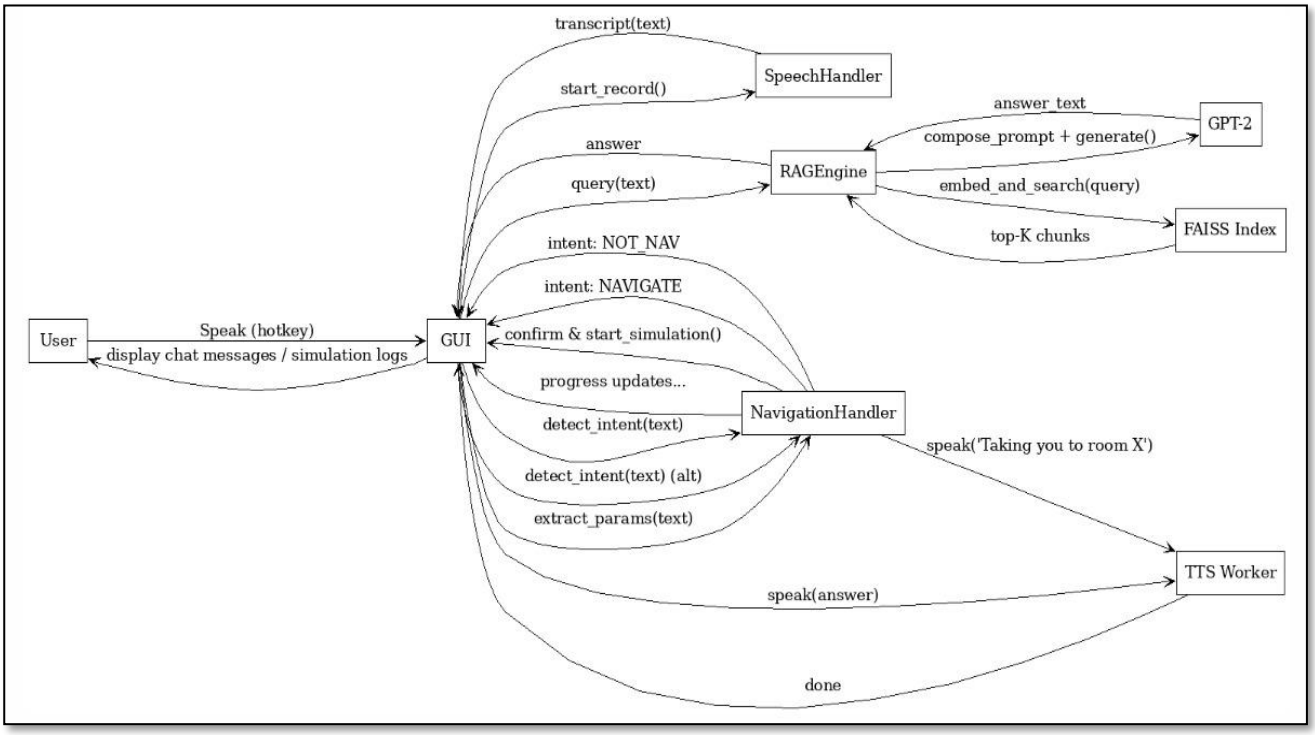

Figure 5.7 RAG Sequence Diagram

Activity Diagram:

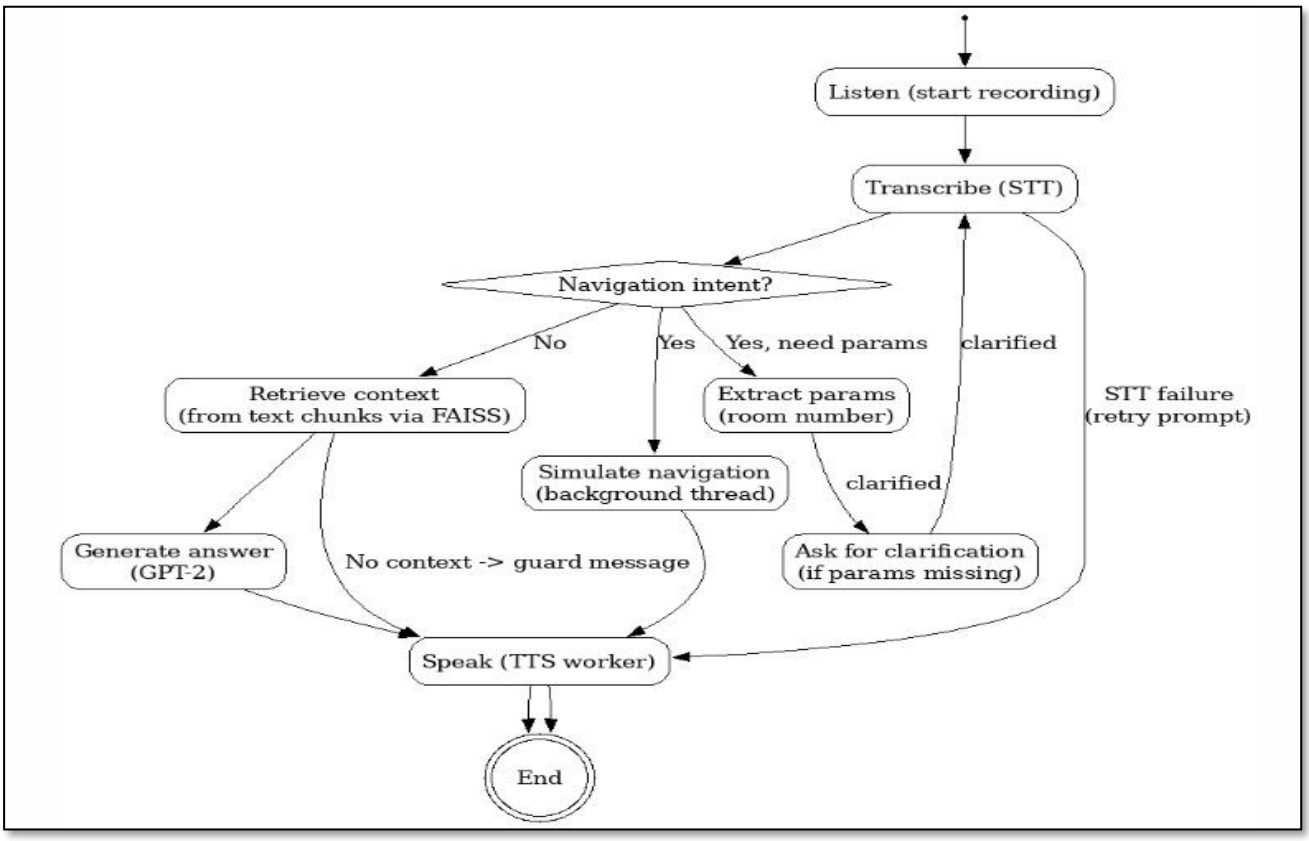

Figure 5.8 RAG Activity Diagram

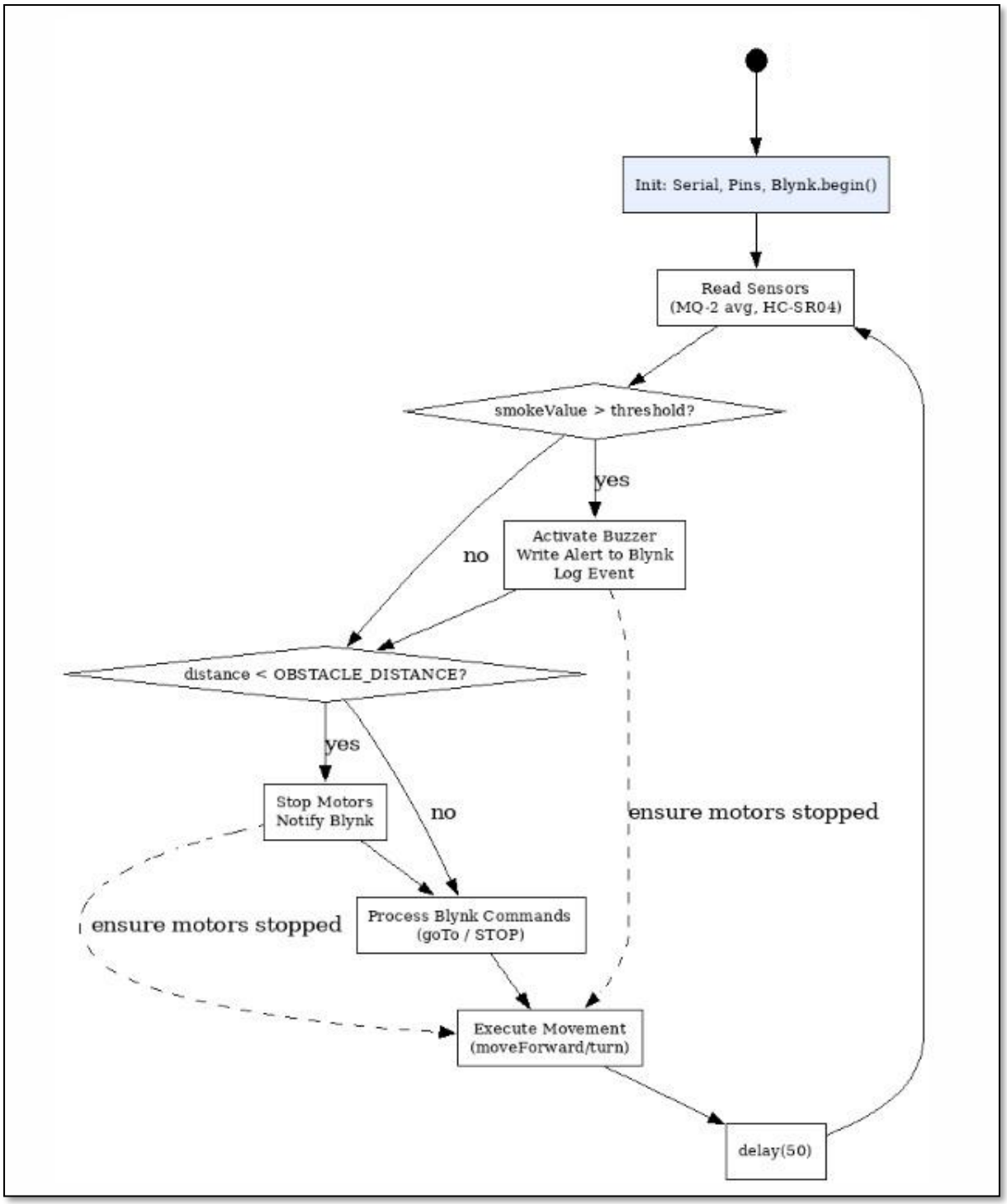

Figure 5.9 RoomRover Activity Diagram

Algorithm

Input: Command / Environmental Data (Obstacle & Smoke Sensor Readings) **Output:** Robot Action / Voice Response / IoT Alerts

Step 1: Voice Input and Intent Recognition

Capture user voice input through microphone. Convert speech to text using the SpeechRecognition module. Apply intent detection to classify command type: **Navigation Command** (e.g., "Move forward", "Turn left"), **Information Query** (e.g., "What is RoomRover?"). If command = "Navigation", forward to Blynk Cloud; else, pass to RAG Engine for response generation.

Step 2: RAG-Based Information Retrieval

The input query is embedded using **SentenceTransformer** or **FAISS vector search**. Retrieve top-k relevant text chunks from local knowledge base. Pass retrieved context and user query to **GPT-2 (Transformer-based generator)** for contextual response synthesis. Display and vocalize response using **Text-to-Speech (pyttsx3)**.

Step 3: IoT Command Transmission

Navigation command is formatted and sent via **Blynk** to the ESP32 microcontroller. Blynk Cloud acts as the intermediary between desktop AI assistant and physical robot. The ESP32 receives command packets and executes corresponding motor actions (forward, left, right, stop).

Step 4: Autonomous Robot Operation (ESP32 Layer)

Continuously read sensor data: **Ultrasonic sensor** → obstacle distance, **MQ2 sensor** → smoke concentration. If smoke detected or obstacle threshold < limit: Trigger **buzzer** and send alert to **Blynk Cloud**, Override motion commands to perform safety stop. Update real-time status (distance, smoke, motion state) on Blynk dashboard.

Step 5: Feedback and Monitoring

Blynk mobile/desktop app displays: Robot movement status, Smoke detection alerts, Connection health and telemetry data User may override movement or send manual commands from app. System continuously synchronizes cloud data with local voice assistant for contextual awareness.

Key Components of the Proposed Framework

1. **ESP32 Control Unit:** Handles motor control, sensor data acquisition, and Wi-Fi communication with Blynk Cloud.
2. **Sensor Suite:** Ultrasonic (HC-SR04) for obstacle detection, MQ2 for smoke/fire detection.
3. **Blynk IoT Cloud:** Middleware enabling two-way data flow between robot and AI assistant.
4. **Motor Driver (L293D):** Controls robot movement based on ESP32 GPIO signals.
5. **Voice Assistant :** Tkinter interface with microphone and speaker support, Integrates Speech-to-Text and Text-to-Speech modules.
6. **RAG Engine (Retrieval-Augmented Generation):** Local vector-based search (FAISS) over text dataset, GPT-2 generates context-aware responses.
7. **Navigation Simulation Module:** Converts recognized commands into simulated movement visuals or control packets.
8. **Cloud Communication Layer:** Enables hybrid operation — local AI decisions + cloud-mediated robot control.

Analysis

1. Performance: ESP32 provides low-latency sensor readings and real-time IoT communication. The RAG engine achieves faster and more contextually accurate responses than standalone GPT-2. Voice interaction latency kept below 2 seconds on standard CPU systems.

2. Scalability: Modular design allows independent scaling of robot and RAG components, Blynk Cloud supports multiple devices and dashboards for multi-robot setups. RAG dataset easily extendable to new domains (campus info, room directories, etc.).

3. Intelligence and Realism: Combination of RAG + intent detection gives both factual recall and conversational fluidity. Simulated navigation replicates real movement for testing without physical robot. AI assistant maintains contextual awareness between multiple commands.

4. System Robustness IoT-based alerts ensure safety even when the assistant is offline. Local voice assistant functions without Internet, ensuring availability. Modular architecture prevents one subsystem failure from halting the other.

5. Limitations: Real robot navigation currently simulated; requires physical path planning for deployment. RAG dataset limited to textual knowledge; lacks visual understanding. Dependent on clear voice input for accurate intent detection.

Chapter 6
Experimental Setup

Software Requirements:-

Programming Languages:

- Python 3.10+ – for Voice Assistant, RAG System, and GUI
- C/C++ (Arduino) – for ESP32 robot firmware

Frameworks:

- PyTorch 2.2.0 – for implementing the RAG (Retrieval-Augmented Generation) model
- Tkinter – for GUI-based desktop interface
- Blynk IoT Platform – for real-time cloud communication between robot and assistant

Supporting Libraries:

- SpeechRecognition, PyAudio – for capturing and processing user voice input
- Pyttsx3 – for text-to-speech output
- Transformers (Hugging Face GPT-2) – for local language generation
- FAISS – for vector-based semantic retrieval in RAG
- SentenceTransformers – for embedding and similarity computation
- Requests, JSON – for communication with Blynk Cloud
- NumPy, Pandas – for data processing and handling
- Tkinter, CustomTkinter – for creating GUI dashboard
- Matplotlib / Plotly – for real-time visualization (optional)
- tqdm – for displaying progress during data preparation or retrieval

External Platforms:

- Blynk Cloud – for sending/receiving robot telemetry and commands
- OpenAI Transformers / Local GPT Models – for RAG text generation

Hardware Requirements: -

Robot Hardware (IoT Subsystem):

- Microcontroller: ESP32 (Wi-Fi enabled)
- Sensors: Ultrasonic Sensor (HC-SR04) – obstacle detection. MQ2 Gas Sensor – smoke/fire detection
- Motor Driver: L293D (dual H-bridge)
- Motors: 2 × DC gear motors
- Buzzer: Active buzzer for alert indication
- Power Supply: 9V–12V DC battery pack

Voice Assistant Hardware (AI Subsystem):

- Processor: Intel Core i5 / Ryzen 5 or higher
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: 256 GB SSD (for models, datasets, and logs)
- Microphone & Speakers: Required for voice input/output
- Network: Stable Wi-Fi connection (for ESP32–Blynk sync)

Operating Environment:-

- **Operating System:** Windows 10 / 11 (64-bit)
- **Environment Type:** Conda / Virtualenv isolated Python environment
- **Connectivity:** Wi-Fi network for ESP32–Cloud–Desktop communication. Optional offline mode for local RAG and voice operations
- **IoT Cloud Platform:** Blynk Cloud (via REST + mobile app)
- **Execution Mode:** Hybrid — local AI processing + remote IoT control

Development Environment:-

- IDE / Code Editor: Visual Studio Code / PyCharm
- Firmware IDE: Arduino IDE (for ESP32)
- Version Control: Git & GitHub (source code management)
- Model Training Platform: Local CPU/GPU (for fine-tuning RAG)
- GUI Toolkit: Tkinter (Python) for desktop interface
- Visualization Tools: Matplotlib, Blynk Dashboard (mobile/web)
- Testing Framework: Pytest / Manual validation via Blynk App
- Communication Testing Tools: Serial Monitor, Blynk Console

Machine Learning & NLP Libraries:-

- **PyTorch:** Deep learning framework for RAG architecture
- **Transformers (Hugging Face):** GPT-2 and tokenizer utilities
- **FAISS:** High-speed vector similarity search for retrieval
- **SentenceTransformers:** Embedding generation for knowledge base indexing
- **NumPy, Pandas:** Numerical and structured data operations
- **SpeechRecognition & PyAudio:** Speech-to-text module
- **Pyttsx3:** Text-to-speech engine for voice response
- **Tkinter:** For creating GUI and user interaction window
- **Requests:** To interact with Blynk Cloud
- **tqdm:** To show progress during indexing or inference
- **YAML:** For loading configuration files
- **JSON:** For structured data exchange

Chapter 7

Project Planning

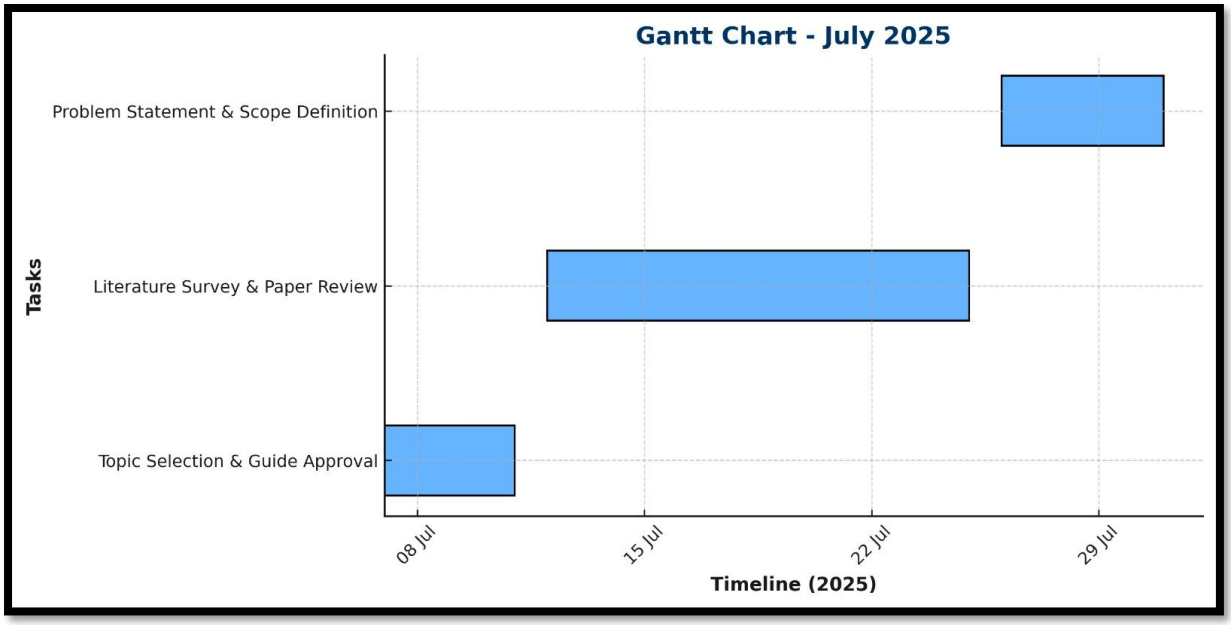

Figure 7.1 July (Gantt Chart)

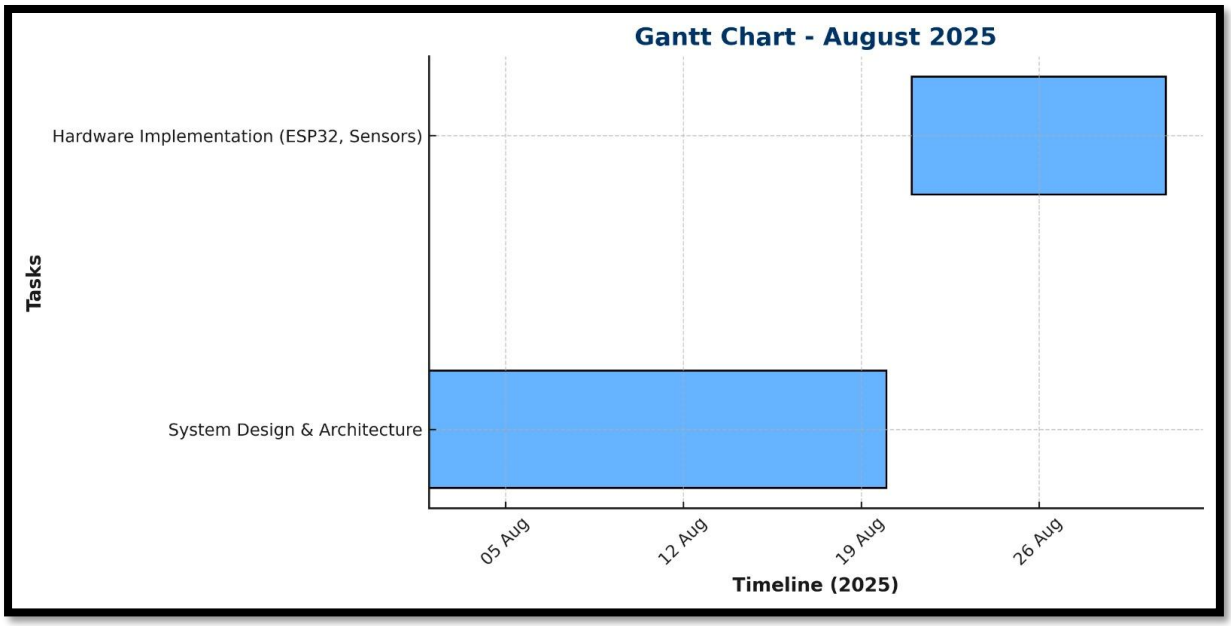

Figure 7.2 August (Gantt Chart)

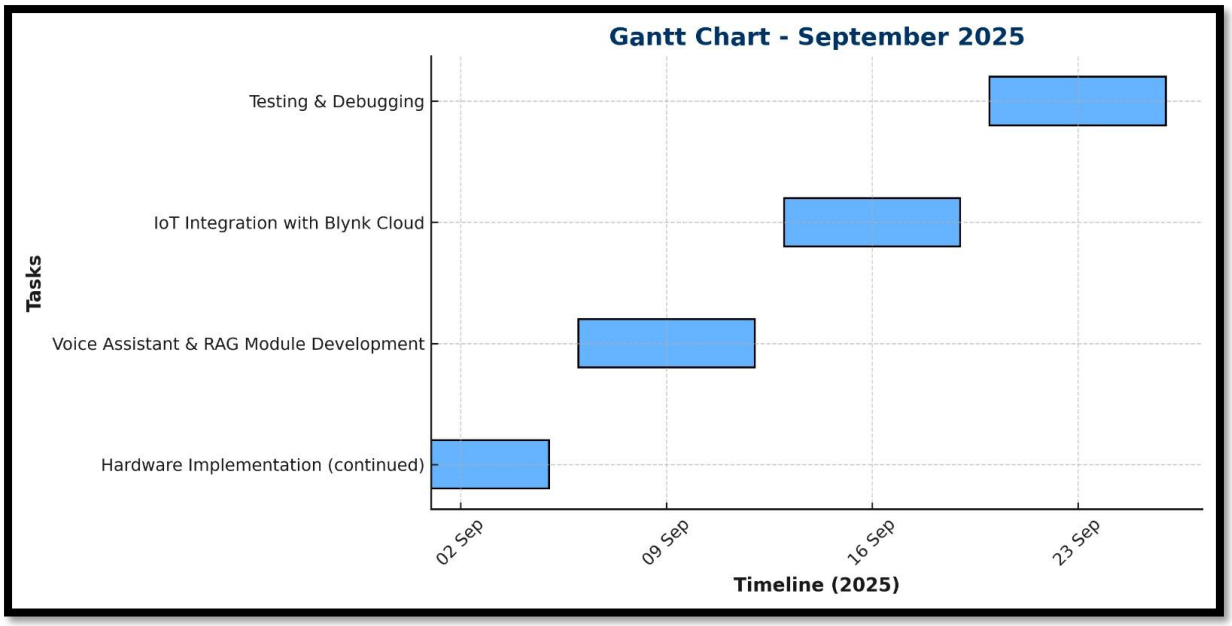

Figure 7.3 September (Gantt Chart)

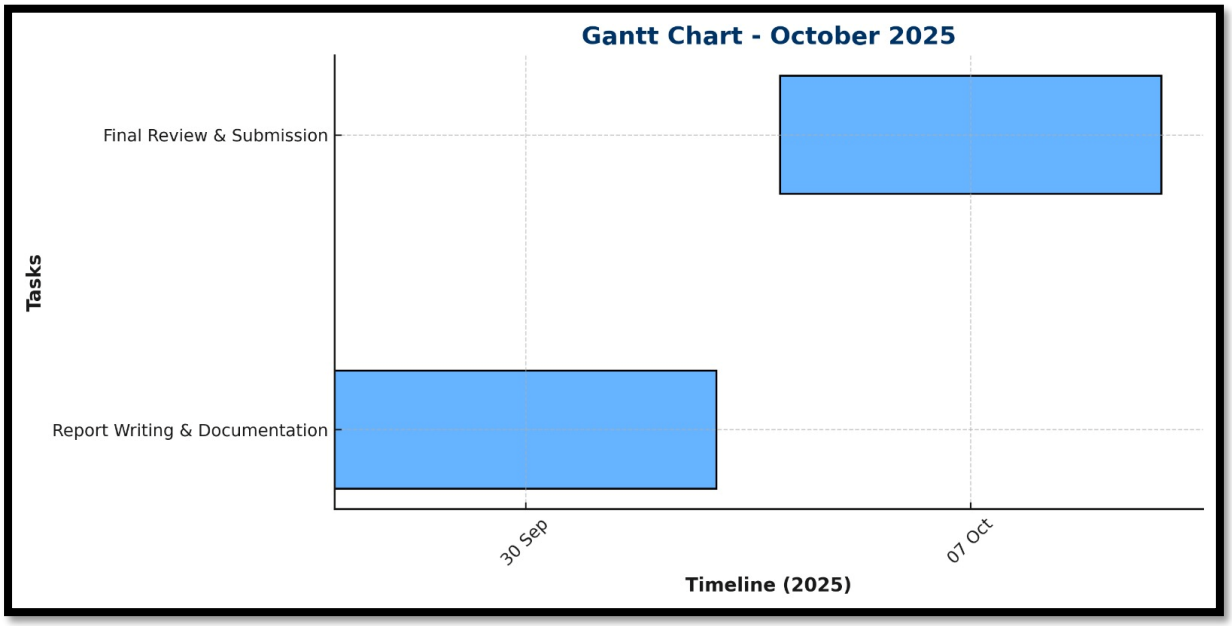

Figure 7.4 October (Gantt Chart)

Chapter 8

Implementation

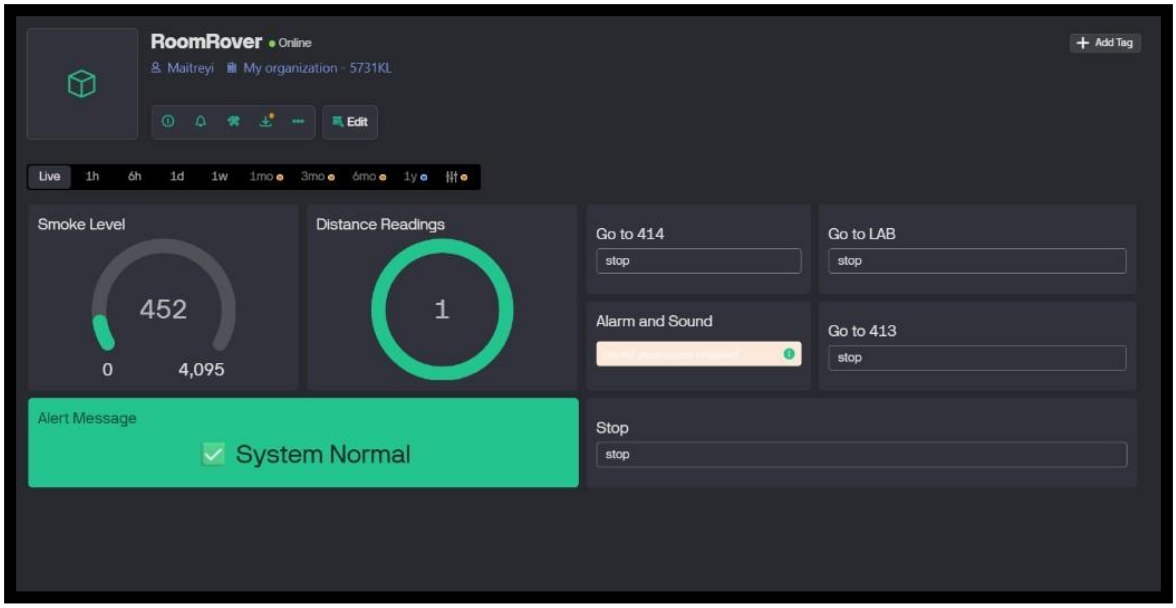

Figure 8.1 Blynk Console

Figure 8.2 Floor Plan

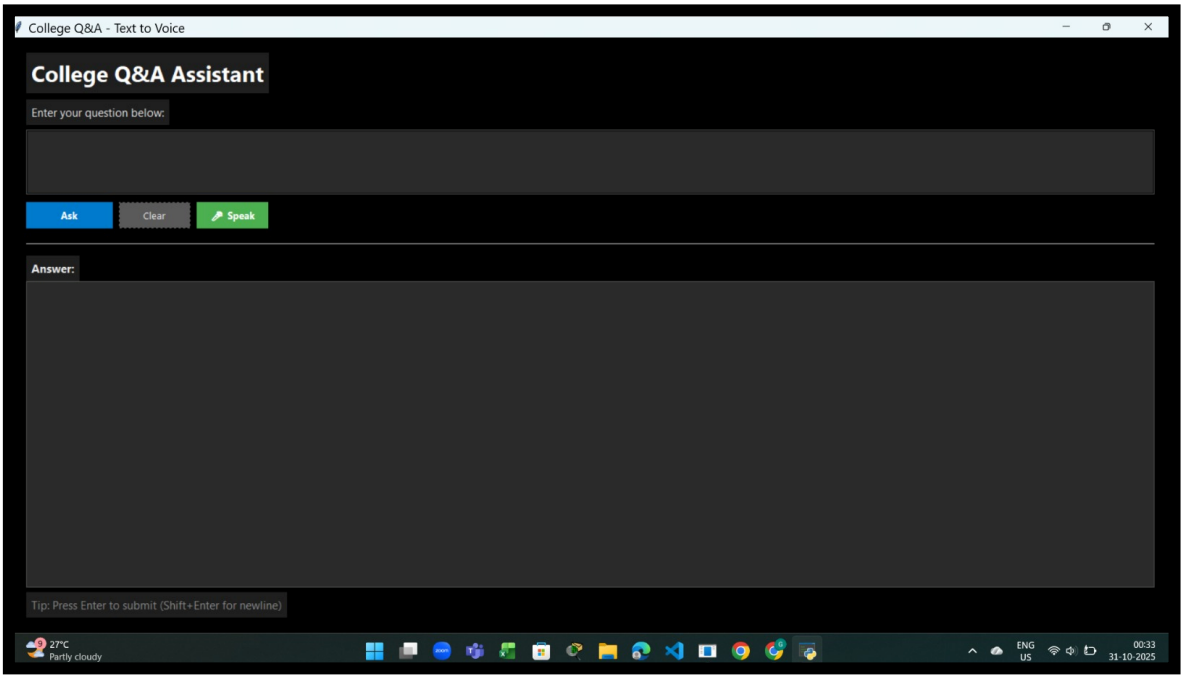

Figure 8.3 Chatbot UI

REFERENCES

[1] Reddy, V. K. G. N., Prajwal, V., Reddy, V. G. N., Srushti, G. S., & Nirmala, S. (2024). *Plan–MORPH: Conversion of 2D Floorplans to 3D Models*. **International Journal of Enhanced Research in Science, Technology & Engineering, 13**(12), 37–42.

[2] Buehler, M. J. (2022). *Generating 3D architectured nature-inspired materials and granular media using diffusion models based on language cues*. **Oxford Open Materials Science, 2**(1), itac010.

[3] Park, S., & Kim, H. (2021). *3DPlanNet: Generating 3D Models from 2D Floor Plan Images Using Ensemble Methods*. **Electronics, 10**(22), 2729.

[4] Juan, Y.-K., Chen, H.-H., & Chi, H.-Y. (2018). *Developing and Evaluating a Virtual Reality-Based Navigation System for Pre-Sale Housing Sales*. **Applied Sciences, 8**(6), 952.

[5] Deaky, B. A., & Parv, A. L. (2018). *Virtual Reality for Real Estate – A Case Study*. **IOP Conference Series: Materials Science and Engineering, 399**, 012013.